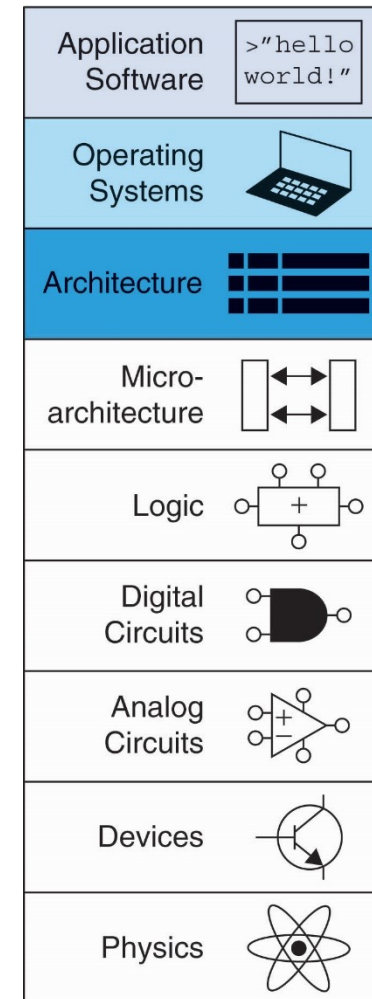


ARM assembly

Architettura: descrizione operativa di computer

- Come un programmatore *vede* a basso livello un computer
- Definisce un insieme di istruzioni e di registri che fungono da operandi per tali istruzioni
- Istruzioni ARM
 - **Assembly language:** formato human-readable
 - **Machine language:** formato computer-readable
- Abbiamo già visto nelle lezioni precedenti come le diverse tipologie di istruzioni (memory, data-processing, e branch) sono tradotte in formato macchina



Locazione degli Operandi

- Operandi di una istruzione ARM
 - Registri
 - Costanti (*immediates*)
- ARM ha 16 registri a 32 bit (R0... R15) che sono fisicamente equivalenti fra loro, ma dal punto di vista logico sono usati con scopi specifici

Name	Use
R0	Argument / return value / temporary variable
R1-R3	Argument / temporary variables
R4-R11	Saved variables
R12	Temporary variable
R13 (SP)	Stack Pointer
R14 (LR)	Link Register
R15 (PC)	Program Counter

Generazione di costanti

E' possibile definire costanti con l'istruzione MOV:

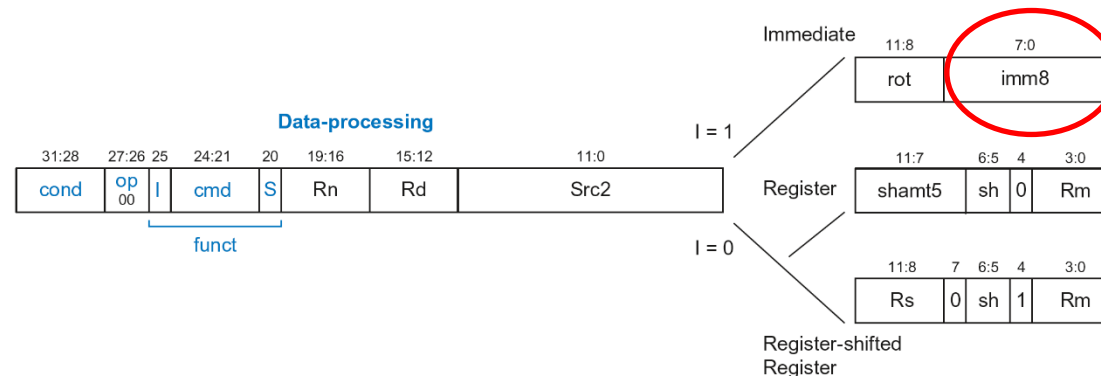
C Code

```
//int: 32-bit signed word  
int a = 23;  
int b = 0x45;
```

ARM Assembly Code

```
; R0 = a, R1 = b  
MOV R0, #23  
MOV R1, #0x45
```

Le costanti così generate hanno una precisione massima di 8 bits



Generazione di costanti

E' possibile definire costanti con l'istruzione MOV:

C Code

```
//int: 32-bit signed word  
int a = 23;  
int b = 0x45;
```

ARM Assembly Code

```
; R0 = a, R1 = b  
MOV R0, #23  
MOV R1, #0x45
```

Nota: MOV può anche essere usato per spostare il contenuto di un registro in un altro registro:

```
MOV R7, R9
```

Generazione di costanti

Generare costanti usando MOV e ORR:

C Code

```
int a = 0x7EDC8765;
```

ARM Assembly Code

```
# R0 = a
MOV R0, #0x7E000000
ORR R0, R0, #0xDC0000
ORR R0, R0, #0x8700
ORR R0, R0, #0x65
```

Istruzioni condizionali

C Code

ARM Assembly Code

```
;R0=f, R1=g, R2=h, R3=i, R4=j
```

```
if (i == j)      CMP R3, R4      ; set flags with R3-R4
    f = g + h;    BNE L1         ; if i!=j, skip if block
                    ADD R0, R1, R2 ; f = g + h

                    L1
f = f - i;        SUB R0, R0, R3  ; f = f - i
```

Nota: il codice assembly effettua il test opposto ($i \neq j$) rispetto a quello di alto livello ($i == j$)

Istruzioni condizionali

C Code

```
if (i == j)
    f = g + h;
f = f - i;
```

ARM Assembly Code

```
;R0=f, R1=g, R2=h, R3=i, R4=j
```

```
CMP    R3, R4        ; set flags with R3-R4
ADDEQ  R0, R1, R2     ; if (i==j) f = g + h
SUB    R0, R0, R2     ; f = f - i
```

Istruzioni condizionali

Codice alternativo per piccoli blocchi di codice:

Original

```
CMP R3, R4
BNE L1
ADD R0, R1, R2
L1
SUB R0, R0, R3
```

Alternate Assembly Code

```
;R0=f, R1=g, R2=h, R3=i, R4=j

CMP    R3, R4        ; set flags with R3-R4
ADDEQ  R0, R1, R2     ; if (i==j) f = g + h
SUB     R0, R0, R3    ; f = f - i
```

Istruzioni condizionali

C Code

```
if (i == j)
    f = g + h;
else
    f = f - i;
```

ARM Assembly Code

```
;R0=f, R1=g, R2=h, R3=i, R4=j

CMP R3, R4      ; set flags with R3-R4
BNE L1          ; if i!=j, skip if block
ADD R0, R1, R2  ; f = g + h
B L2            ; branch past else block

L1
SUB R0, R0, R3   ; f = f - i
L2
```

Istruzioni condizionali

Codice alternativo per piccoli blocchi di codice:

Original

```
CMP R3, R4
BNE L1
ADD R0, R1, R2
B    L2
L1
  SUB R0, R0, R3
L2
```

Alternate Assembly Code

```
;R0=f, R1=g, R2=h, R3=i, R4=j

CMP    R3, R4      ; set flags with R3-R4
ADDEQ  R0, R1, R2   ; if (i==j) f = g + h

SUBNE  R0, R0, R3   ; else f = f - i
```

Istruzioni iterative

C Code

```
// determines the power  
// of x such that 2x = 128  
int pow = 1;  
int x    = 0;
```

```
while (pow != 128) {  
  
    pow = pow * 2;  
    x = x + 1;  
}
```

ARM Assembly Code

```
; R0 = pow, R1 = x  
MOV    R0, #1          ; pow = 1  
MOV    R1, #0          ; x = 0  
  
WHILE  
    CMP R0, #128        ; R0-128  
    BEQ DONE            ; if (pow==128)  
                        ; exit loop  
    LSL R0, R0, #1       ; pow=pow*2  
    ADD R1, R1, #1       ; x=x+1  
    B   WHILE            ; repeat loop  
  
DONE
```

Il codice assembly verifica la condizione opposta ($\text{pow} == 128$) a quella del C ($\text{pow} != 128$).

Istruzioni iterative

**for (initialization; condition; loop operation)
statement**

- **initialization:** eseguita prima che il loop inizi
- **condition:** condizione di continuazione che è verificata all'inizio di ogni iterazione
- **loop operation:** eseguita alla fine di ogni iterazione
- **statement:** eseguito ad ogni iterazione, ovvero fintantoché la condizione di continuazione è verificata

Istruzioni iterative

C Code

```
// adds numbers from 1-9
int sum = 0;
int i;
```

```
for (i=1; i!=10; i=i+1)
    sum = sum + i;
```

ARM Assembly Code

```
; R0 = i, R1 = sum
MOV R1, #0           ; sum = 0
MOV R0, #1           ; i = 1

FOR
    CMP R0, #10       ; R0-10
    BEQ DONE          ; if (i==10)
                        ; exit loop

    ADD R1, R1, R0     ; sum=sum + i
    ADD R0, R0, #1     ; i = i + 1
    B FOR              ; repeat loop

DONE
```

Istruzioni iterative

In ARM, i loop decrescenti fino a 0 sono più efficienti

C Code

```
// adds numbers from 1-9
int sum = 0;
int i;
```

```
for (i=9; i!=0; i=i-1)
    sum = sum + i;
```

ARM Assembly Code

```
; R0 = i, R1 = sum
MOV R1, #0           ; sum = 0
MOV R0, #9           ; i = 9

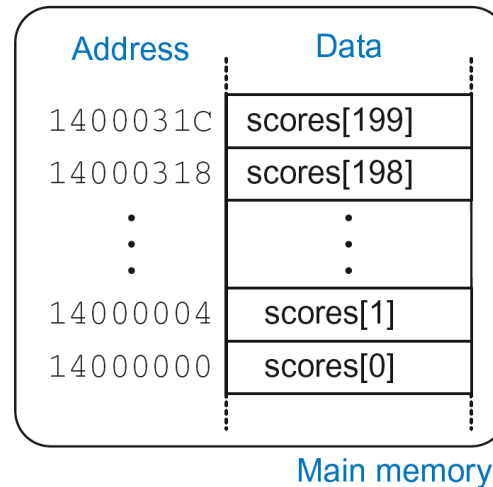
FOR
    ADD R1, R1, R0    ; sum=sum + i
    SUBS R0, R0, #1   ; i = i - 1
                        ; and set flags
    BNE FOR           ; if (i!=0)
                        ; repeat loop
```

Si risparmiano 2 istruzioni per ogni iterazione:

- Si accorpano decremento e comparazione: SUBS R0, R0, #1
- Solo un branch invece di due
- Nota l'uso di NE che corrisponde a $\bar{Z}$, ovvero R0 non è ancora 0

Arrays

- Consente l'accesso ad una quantità di dati simili
 - **Index:** accesso ad un qualche elemento
 - **Size:** numero di elementi
- Esempio: array di parole (32bit)
 - **Base address** = 0x14000000 (indirizzo di scores[0])
 - Gli altri elementi sono raggiungibili a partire dal base address



Arrays

C Code

```
int array[5];  
array[0] = array[0] * 8;  
array[1] = array[1] * 8;
```

ARM Assembly Code

; R0 = array base address

```
MOV R0, #0x60000000      ; R0 = 0x60000000
```

```
LDR R1, [R0]             ; R1 = array[0]
```

```
LSL R1, R1, 3             ; R1 = R1 << 3 = R1*8
```

```
STR R1, [R0]             ; array[0] = R1
```

```
LDR R1, [R0, #4]         ; R1 = array[1]
```

```
LSL R1, R1, 3             ; R1 = R1 << 3 = R1*8
```

```
STR R1, [R0, #4]         ; array[1] = R1
```

Arrays

C Code

```
int array[200];
int i;

for (i=199; i >= 0; i = i - 1)
    array[i] = array[i] * 8;
```

ARM Assembly Code

```
; R0 = array base address, R1 = i
MOV R0, 0x60000000
MOV R1, #199

FOR
    LDR R2, [R0, R1, LSL #2]      ; R2 = array(i)
    LSL R2, R2, #3                ; R2 = R2<<3 = R2*8
    STR R2, [R0, R1, LSL #2]      ; array(i) = R2
    SUBS R1, R1, #1               ; i = i - 1
                                   ; and set flags
    BPL FOR                       ; if (i>=0) repeat loop
```

```
LDR R2, [R0, R1, LSL #2]
```

Poiché gli elementi degli array sono degli interi, ovvero parole di 4 byte, gli indirizzi devono scorrere di 4 in 4

Chiamate di funzioni

- **Caller:** funzione chiamante, in questo caso `main`
- **Callee:** funzione chiamata, in questo caso `sum`

C Code

```
void main()
{
    int y;
    y = sum(42, 7);
    ...
}

int sum(int a, int b)
{
    return (a + b);
}
```

Contratto

- Caller:

- Passa gli argomenti al callee
- Esegue un jump al callee

- Callee:

- Esegue la funzione chiamata
- ritorna il risultato al caller
- ritorna nel punto di chiamata del caller
- non deve sovrascrivere i saved register e la memoria usata dal caller

Convenzioni ARM per chiamata a funzione

- **Chiamata a funzione:** branch and link BL
- **Return** da funzione: ripristina in PC il valore del link register
MOV PC, LR
- **Argomenti:** R0–R3
- **Valore di ritorno:** R0

Esempio di chiamata a funzione

C Code

```
int main() {  
    simple();  
    a = b + c;  
}
```

```
void simple() {  
    return;  
}
```

ARM Assembly Code

```
0x00000200 MAIN      BL  SIMPLE  
0x00000204           ADD R4, R5, R6  
...
```

```
0x00401020 SIMPLE    MOV PC, LR
```

void significa che **simple** non ritorna un valore

Esempio di chiamata a funzione

C Code

```
int main() {  
    simple();  
    a = b + c;  
}  
  
void simple() {  
    return;  
}
```

ARM Assembly Code

```
0x00000200 MAIN      BL  SIMPLE  
0x00000204          ADD R4, R5, R6  
...
```

```
0x00401020 SIMPLE   MOV PC, LR
```

BL

branches to SIMPLE

LR = 0x00000204

MOV PC, LR

makes PC = LR

(the next instruction executed is at 0x00000204)

Argomenti e valore di ritorno

Convenzioni ARM:

- Argomenti: R0 - R3
- Valore di ritorno: R0

C Code

```
int main(){
    int y;
    ...
    y = diffofsums(2, 3, 4, 5); // 4 arguments
    ...
}

int diffofsums(int f, int g, int h, int i){
    int result;
    result = (f + g) - (h + i);
    return result;           // return value
}
```

Argomenti e valore di ritorno

ARM Assembly Code

```
; R4 = y
```

```
MAIN
```

```
...
```

```
MOV R0, #2           ; argument 0 = 2
```

```
MOV R1, #3           ; argument 1 = 3
```

```
MOV R2, #4           ; argument 2 = 4
```

```
MOV R3, #5           ; argument 3 = 5
```

```
BL DIFFOFSUMS        ; call function
```

```
MOV R4, R0            ; y = returned value
```

```
...
```

```
; R4 = result
```

```
DIFFOFSUMS
```

```
ADD R8, R0, R1        ; R8 = f + g
```

```
ADD R9, R2, R3        ; R9 = h + i
```

```
SUB R4, R8, R9        ; result = (f + g) - (h + i)
```

```
MOV R0, R4            ; put return value in R0
```

```
MOV PC, LR            ; return to caller
```

Argomenti e valore di ritorno

ARM Assembly Code

```
; R4 = result
DIFFOFSUMS
    ADD R8, R0, R1      ; R8 = f + g
    ADD R9, R2, R3      ; R9 = h + i
    SUB R4, R8, R9      ; result = (f + g) - (h + i)
    MOV R0, R4           ; put return value in R0
    MOV PC, LR           ; return to caller
```

- `diffofsums` sovrascrive indebitamente i 3 registri R4, R8, R9 dedicati alle saved variables della funzione chiamante `main`
- `diffofsums` invece dovrebbe usare lo *stack* in memoria per memorizzare temporaneamente i valori di questi registri prima di operare su di essi

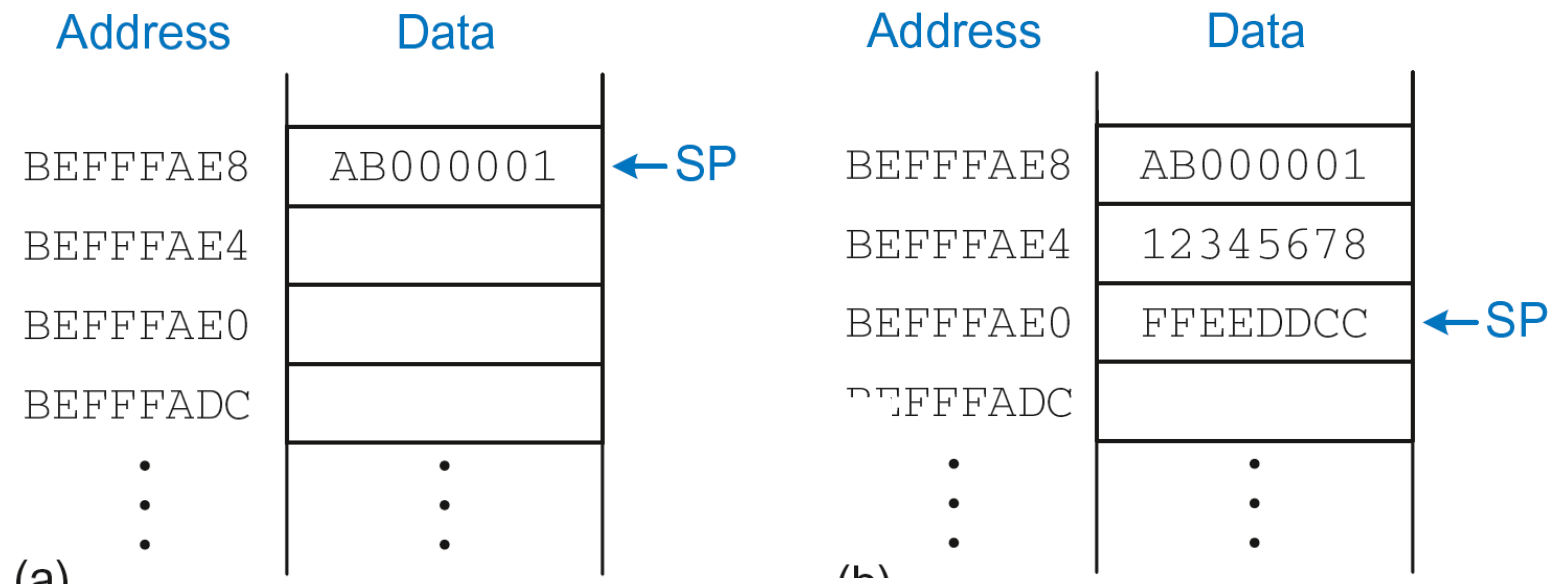
Lo stack delle funzioni

- Lo stack costituisce la memoria usata per salvare temporaneamente il valore delle variabili
- E' organizzato come una pila last-in-first-out (LIFO)
- Operazioni sullo stack
 - **Expands:** usa più memoria quando necessaria (push)
 - **Contracts:** usa meno memoria quando possibile (pop)



Lo stack delle funzioni

- In termini di indirizzi di memoria, l'espansione dello stack avviene in senso decrescente (dagli indirizzi maggiori a quelli più piccoli)
- Stack pointer: SP *punta* al top dello stack



Stack expands by 2 words

Come le funzioni usano lo stack

- Le funzioni chiamate non devono avere side effect inattesi
- Invece, nell'esempio precedente diffofsums sovrascrive indebitamente i 3 registri R4, R8, R9

ARM Assembly Code

```
; R4 = result
```

```
DIFFOFSUMS
```

ADD R8 , R0, R1	; R8 = f + g
ADD R9 , R2, R3	; R9 = h + i
SUB R4 , R8, R9	; result = (f + g) - (h + i)
MOV R0, R4	; put return value in R0
MOV PC, LR	; return to caller

Memorizzare i registri sullo stack

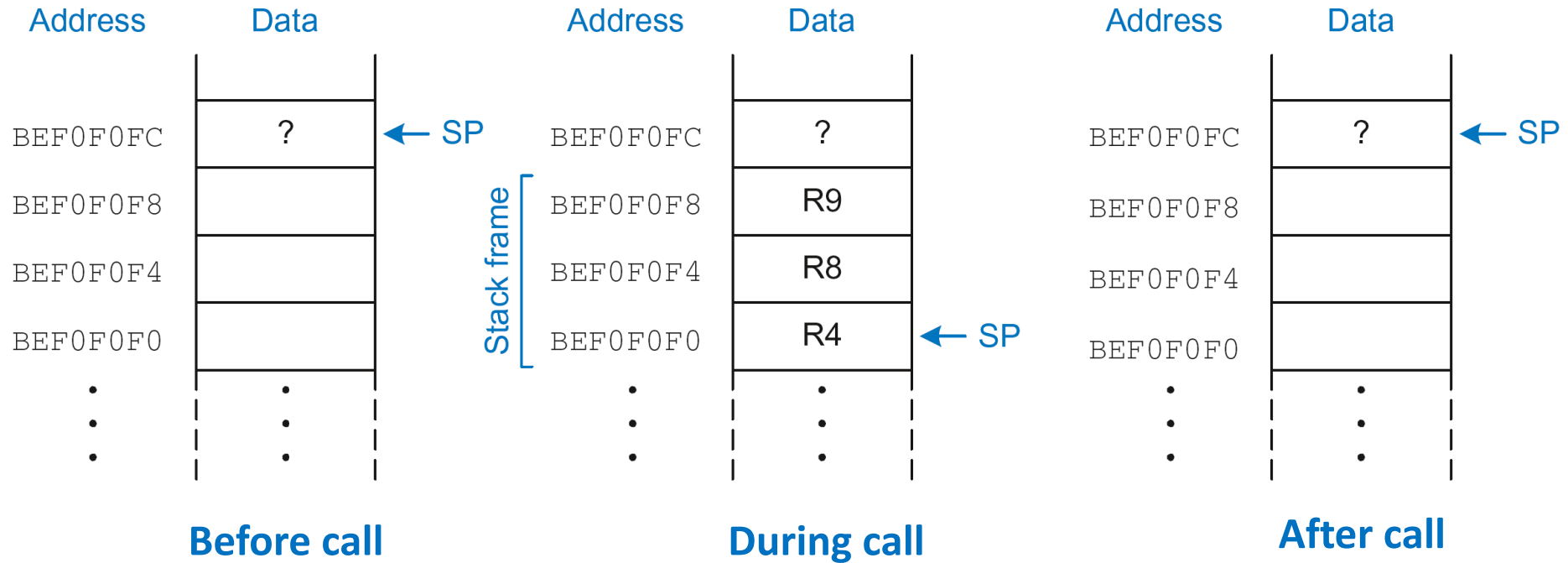
ARM Assembly Code

; R2 = result

DIFFOFSUMS

```
SUB SP, SP, #12      ; make space on stack for 3 registers
STR R9, [SP, #8]     ; save R9 on stack
STR R8, [SP, #4]     ; save R8 on stack
STR R4, [SP]         ; save R4 on stack
ADD R8, R0, R1        ; R8 = f + g
ADD R9, R2, R3        ; R9 = h + i
SUB R4, R8, R9        ; result = (f + g) - (h + i)
MOV R0, R4            ; put return value in R0
LDR R4, [SP]         ; restore R4 from stack
LDR R8, [SP, #4]     ; restore R8 from stack
LDR R9, [SP, #8]     ; restore R9 from stack
ADD SP, SP, #12     ; deallocate stack space
MOV PC, LR           ; return to caller
```

Memorizzare i registri sullo stack



Callee/caller saved

Preserved <i>Callee-Saved</i>	Nonpreserved <i>Caller-Saved</i>
R4-R11	R12
R14 (LR)	R0-R3
R13 (SP)	CPSR
stack above SP	stack below SP

Pop e push

- Salvare e ripristinare registri dallo stack è una operazione così frequente che ARM mette a disposizione delle istruzioni specifiche, Pop e Push, che consentono di avere un codice più succinto.
- Push consente di salvare in memoria più registri aggiornare consistentemente lo stack: `PUSH {R4, R8, R9}`
- Pop ripristina uno o più registri e incrementa consistentemente il valore dello stack: `POP {R4, R8, R9}`

Chiamate a funzioni innestate

C Code

```
int f1(int a, int b) {
    int i, x;
    x = (a + b) * (a - b);
    for (i=0; i<a; i++)
        x = x + f2(b+i);
    return x;
}

int f2(int p) {
    int r;
    r = p + 5;
    return r + p;
}
```

ARM Assembly Code

```
; R0=a, R1=b, R4=i, R5=x
```

```
F1
```

```
    PUSH {R4, R5, LR}
```

```
    ADD   R5, R0, R1
```

```
    SUB   R12, R0, R1
```

```
    MUL   R5, R5, R12
```

```
    MOV   R4, #0
```

```
FOR
```

```
    CMP   R4, R0
```

```
    BGE   RETURN
```

```
    PUSH {R0, R1}
```

```
    ADD   R0, R1, R4
```

```
    BL    F2
```

```
    ADD   R5, R5, R0
```

```
    POP   {R0, R1}
```

```
    ADD   R4, R4, #1
```

```
    B     FOR
```

```
RETURN
```

```
    MOV   R0, R5
```

```
    POP   {R4, R5, LR}
```

```
    MOV   PC, LR
```

```
; R0=p, R4=r
```

```
F2
```

```
    PUSH {R4}
```

```
    ADD   R4, R0, 5
```

```
    ADD   R0, R4, R0
```

```
    POP   {R4}
```

```
    MOV   PC, LR
```

Chiamate a funzioni innestate

ARM Assembly Code

```
; R0=a, R1=b, R4=i, R5=x
```

```
F1
```

```
    PUSH {R4, R5, LR} ; save regs
    ADD  R5, R0, R1    ; x = (a+b)
    SUB  R12, R0, R1   ; temp = (a-b)
    MUL  R5, R5, R12   ; x = x*temp
    MOV  R4, #0        ; i = 0
```

```
FOR
```

```
    CMP  R4, R0        ; i < a?
    BGE  RETURN        ; no: exit loop
    PUSH {R0, R1}      ; save regs
    ADD  R0, R1, R4     ; arg is b+i
    BL   F2            ; call f2(b+i)
    ADD  R5, R5, R0     ; x = x+f2(b+i)
    POP  {R0, R1}      ; restore regs
    ADD  R4, R4, #1     ; i++
    B    FOR           ; repeat loop
```

```
RETURN
```

```
    MOV  R0, R5        ; return x
    POP  {R4, R5, LR}  ; restore regs
    MOV  PC, LR        ; return
```

```
; R0=p, R4=r
```

```
F2
```

```
    PUSH {R4}          ; save regs
    ADD  R4, R0, #5     ; r = p+5
    ADD  R0, R4, R0     ; return r+p
    POP  {R4}          ; restore regs
    MOV  PC, LR        ; return
```

Address	Data	
BEF7FF0C	?	← SP
BEF7FF08		
BEF7FF04		
BEF7FF00		
BEF7FEFC		
BEF7FEF8		
BEF7FEF4		
⋮	⋮	
⋮	⋮	
⋮	⋮	

At beginning of f1

Address	Data	
BEF7FF0C	?	
BEF7FF08	LR	
BEF7FF04	R5	
BEF7FF00	R4	
BEF7FEFC	R1	
BEF7FEF8	R0	← SP
BEF7FEF4		
⋮	⋮	
⋮	⋮	
⋮	⋮	

Just before calling f2

Address	Data	
BEF7FF0C	?	
BEF7FF08	LR	
BEF7FF04	R5	
BEF7FF00	R4	
BEF7FEFC	R1	
BEF7FEF8	R0	
BEF7FEF4	R4	← SP
⋮	⋮	
⋮	⋮	
⋮	⋮	

After calling f2

C Code

```
int factorial(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return (n * factorial(n-1));  
}
```

ARM Assembly Code

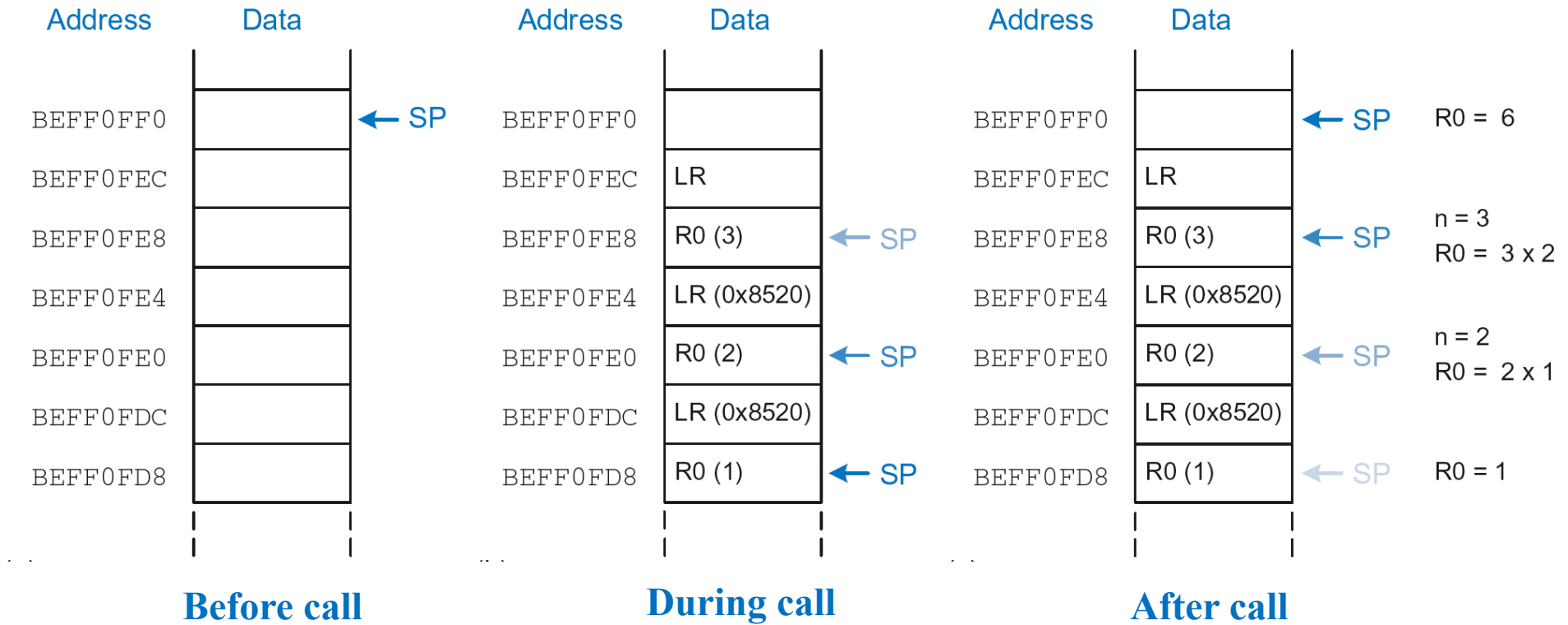
```
0x94 FACTORIAL  STR R0, [SP, #-4]!    ;store R0 on stack
0x98            STR LR, [SP, #-4]!    ;store LR on stack
0x9C            CMP R0, #2            ;set flags with R0-2
0xA0            BHS ELSE              ;if (r0>=2) branch to else
0xA4            MOV R0, #1            ; otherwise return 1
0xA8            ADD SP, SP, #8        ; restore SP 1
0xAC            MOV PC, LR           ; return
0xB0 ELSE      SUB R0, R0, #1        ; n = n - 1
0xB4            BL  FACTORIAL         ; recursive call
0xB8            LDR LR, [SP], #4      ; restore LR
0xBC            LDR R1, [SP], #4      ; restore R0 (n) into R1
0xC0            MUL R0, R1, R0        ; R0 = n*factorial(n-1)
0xC4            MOV PC, LR           ; return
```

C Code

```
int factorial(int n) {  
    if (n <= 1)  
        return 1;  
  
    else  
        return (n * factorial(n-1));  
}
```

ARM Assembly Code

```
0x94 FACTORIAL    STR R0, [SP, #-4]!  
0x98             STR LR, [SP, #-4]!  
0x9C             CMP R0, #2  
0xA0             BHS ELSE  
0xA4             MOV R0, #1  
0xA8             ADD SP, SP, #8  
0xAC             MOV PC, LR  
0xB0 ELSE        SUB R0, R0, #1  
0xB4             BL  FACTORIAL  
0xB8             LDR LR, [SP], #4  
0xBC             LDR R1, [SP], #4  
0xC0             MUL R0, R1, R0  
0xC4             MOV PC, LR
```



Esercizi

cond	Mnemonic	Name	CondEx
0000	EQ	Equal	Z
0001	NE	Not equal	$\overline{Z}$
0010	CS/HS	Carry set / unsigned higher or same	C
0011	CC/LO	Carry clear / unsigned lower	$\overline{C}$
0100	MI	Minus / negative	N
0101	PL	Plus / positive or zero	$\overline{N}$
0110	VS	Overflow / overflow set	V
0111	VC	No overflow / overflow clear	$\overline{V}$
1000	HI	Unsigned higher	$\overline{Z}C$
1001	LS	Unsigned lower or same	$Z \text{ OR } \overline{C}$
1010	GE	Signed greater than or equal	$\overline{N \oplus V}$
1011	LT	Signed less than	$N \oplus V$
1100	GT	Signed greater than	$\overline{Z(N \oplus V)}$
1101	LE	Signed less than or equal	$Z \text{ OR } (N \oplus V)$
1110	AL (or none)	Always / unconditional	Ignored

Esercizi

Si supponga che lo stato dei registri R0, R1, e R2 sia rispettivamente 0x000000100, 0x0000002A, e 0x000000133. Calcolare lo stato dei flag NZCV del Current Program Status Register al termine delle seguenti istruzioni:

CMP R0 R1

SUBGES R0 R0 R2

ADDLTS R0 R1 R2

Esercizi

Si supponga che lo stato dei registri R0, R1, e R2 sia rispettivamente 0x000000100, 0x00000002A, e 0x000000133. Calcolare lo stato dei flag NZCV del Current Program Status Register al termine delle seguenti istruzioni:

```
CMP R0 R1  
SUBGES R0 R0 R2  
ADDLTS R0 R1 R2
```

R0 è maggiore di R1 quindi SUBGES viene eseguita
R0= 0x FFFF FFCD N: 1 Z:0 C:0 V:0

La condizione LT (N XOR V) è soddisfatta quindi ADDLTS viene eseguita
R0= 0x 0000 015D
N: 0 Z:0 C:0 V:0

Esercizi

Si supponga che lo stato dei registri R0, R1, e R2 sia rispettivamente #00000100, #00000100, e #0000010A.
Calcolare lo stato dei flag NZCV del Current Program Status Register al termine delle seguenti istruzioni:

```
CMP R0 R1  
SUBGES R0 R0 R2  
ADDLTS R0 R1 R2
```

Esercizi

Il seguente codice assembly rappresenta la funzione fattoriale, ma ci sono due errori. Scrivere nell'apposito riquadro il codice corretto. [Si ricordi che SP sta per stack pointer, LR per link register e PC per program counter.]

```
FACTORIAL
    PUSH R0, LR
    CMP R0, #1
    BLT ELSE
    MOV R0, #1
    ADD SP, SP, #8
    MOV PC, LR
ELSE
    SUB R0, R0, #1
    BL FACTORIAL
    POP R1, LR
    MUL R0, R1, R0
    MOV PC, SP
```

Esercizi

Il seguente codice assembly rappresenta la funzione fattoriale, ma ci sono due errori. Scrivere nell'apposito riquadro il codice corretto. [Si ricordi che SP sta per stack pointer, LR per link register e PC per program counter.]

```
FACTORIAL
    PUSH R0, LR
    CMP R0, #1
    BLT ELSE
    MOV R0, #1
    ADD SP, SP, #8
    MOV PC, LR
ELSE
    SUB R0, R0, #1
    BL FACTORIAL
    POP R1, LR
    MUL R0, R1, R0
    MOV PC, SP
```

```
FACTORIAL
    PUSH R0, LR
    CMP R0, #1
    BGT ELSE
    MOV R0, #1
    ADD SP, SP, #8
    MOV PC, LR
ELSE
    SUB R0, R0, #1
    BL FACTORIAL
    POP R1, LR
    MUL R0, R1, R0
    MOV PC, LR
```

Esercizi

Il seguente codice assembly rappresenta la funzione fattoriale, ma ci sono due errori. Scrivere nell'apposito riquadro il codice corretto. [Si ricordi che SP sta per stack pointer, LR per link register e PC per program counter.]

```
FACTORIAL
    PUSH R0, LR
    CMP R0, #1
    BGT ELSE
    MOV R0, #1
    ADD SP, SP, #8
    MOV PC, LR
ELSE
    SUB R0, R0, #2
    BL FACTORIAL
    POP R1, LR
    MUL R0, R1, R0
    MOV PC, SP
```

Esercizi

Il seguente codice assembly rappresenta la funzione fattoriale, ma ci sono due errori. Scrivere nell'apposito riquadro il codice corretto. [Si ricordi che SP sta per stack pointer, LR per link register e PC per program counter.]

```
FACTORIAL
    PUSH R0, LR
    CMP R0, #1
    BGT ELSE
    MOV R0, #1
    ADD SP, SP, #8
    MOV PC, LR
ELSE
    SUB R0, R0, #2
    BL FACTORIAL
    POP R1, LR
    MUL R0, R1, R0
    MOV PC, SP
```

```
FACTORIAL
    PUSH R0, LR
    CMP R0, #1
    BGT ELSE
    MOV R0, #1
    ADD SP, SP, #8
    MOV PC, LR
ELSE
    SUB R0, R0, #1
    BL FACTORIAL
    POP R1, LR
    MUL R0, R1, R0
    MOV PC, LR
```

Esercizi

Si consideri il seguente programma assembly:

```
MOVE R0, #5  
MOVE R1, #0x23  
LOOP  
SUBS R1, R1, R0  
ADD R0, R0, #10  
BPL LOOP  
ADD R1, R1, #20
```

Indicare esadecimale il valore di R1 al termine dell'esecuzione.

Esercizi

Si consideri il seguente programma assembly:

```
MOVE R0, #5
MOVE R1, #0x23
LOOP
SUBS R1, R1, R0
ADD R0, R0, #10
BPL LOOP
ADD R1, R1, #20
```

Indicare esadecimale il valore di R1 al termine dell'esecuzione.

R1 35

R0 5

R1 30

R0 15

30>=0?

R1 15

R0 25

15>=0?

R1 -10

R0 35

-10>=0?

Esercizi

Si consideri il seguente programma assembly:

```
MOVE R0, #5
MOVE R1, #0x23
LOOP
CMP R1, R0
BLT DONE
SUB R0, R1, R0
SUB R1, R1, #3
B LOOP
DONE
ADD R1, R1, #10
```

Indicare esadecimale il valore di R1 al termine dell'esecuzione.

Esercizi

R1 35

R0 5

35<5?

R1 32

R0 30

32<30?

R1 29

R0 2

29<2?

R1 26

R0 27

26<27?

R1 36

R1 0x24

Si consideri il seguente programma assembly:

MOVE R0, #5

MOVE R1, #0x23

LOOP

CMP R1, R0

BLT DONE

SUB R0, R1, R0

SUB R1, R1, #3

B LOOP

DONE

ADD R1, R1, #10

Indicare esadecimale il valore di R1 al termine dell'esecuzione.

-8
-12
-16
-20
-24

i
j
f
g
h

FP
1
1
4
3
2

← FP, SP

← SP